Stock Price Analytics and Forecasting using Modern Data Engineering Tools

Rishitha Gogineni
DATA 226: Data Warehousing and Pipeline Management
San José State University

Abstract

This report presents an end-to-end stock price analytics and forecasting system developed using **Snowflake**, **Apache Airflow**, **dbt (Data Build Tool)**, and **Preset (Superset)** for visualization. The project is divided into two phases: **Lab 1** focuses on building the ETL and **Snowflake ML Forecasting** pipelines, while **Lab2** extends the system with dbt-based ELT transformations and BI visualization. The implementation automates data ingestion from the yfinance API for stocks such as **AAPL**, **MSFT**, and **AMZN**, performs data transformation through dbt, and generates 7-day forecasts within Snowflake. The final outputs are analytical tables and interactive dashboards that provide accurate, up-to-date financial insights.

I Problem Statement

This project develops an automated data analytics system to collect, transform, and forecast stock prices for **Apple (AAPL)**, **Microsoft (MSFT)**, and **Amazon (AMZN)** using the yfinance API. **Airflow** orchestrates ETL workflows that extract and load data into **Snowflake**, where the **Snowflake ML Forecast** model predicts the next seven days of prices. **dbt** performs ELT transformations to compute key indicators such as **Moving Averages**, **RSI**, **Momentum**, and **Daily Returns**. Finally, **Preset** visualizes the analytical results through interactive dashboards, enabling fast and reliable stock market insights.

II Solution Requirements and Functional Analysis

The system delivers a fully automated, daily-updated stock analytics and 7+ day price forecasting pipeline for AAPL, MSFT, and AMZN using yfinance, Airflow, Snowflake, dbt, and Preset (Superset). **Functional Requirements**

- **ETL (Lab 1)**: Airflow DAG extracts OHLCV data from yfinance and upserts it into Snowflake table RAW.STOCK_PRICES (idempotent MERGE).
- **Forecasting (Lab 1)**: Second Airflow DAG runs Snowflake Cortex ML FORECAST per ticker, generates predictions with confidence bounds, and unions historical + forecast rows.
- **ELT with dbt (Lab 2)**: Third Airflow DAG triggers dbt to:
  - Clean data in staging (views)
  - Calculate indicators (SMA-20/50/200, RSI-14, daily returns, volatility) in intermediate/ephemeral models
  - Produce single final mart table ANALYTICS.FINAL_PRICES_FORECAST
- **Visualization (Lab 2)**: Preset dashboards connect directly to the final mart table and display price + forecasts, moving averages, RSI, returns, and volatility with ticker/date filters.

Non-Functional Requirements

- All three Airflow DAGs run daily; forecasting and dbt DAGs execute only after successful upstream tasks.
- Design is idempotent and easily extendable to new tickers via Airflow variables. Overall, the system integrates Airflow for orchestration, Snowflake for storage and forecasting, dbt for transformation and analytics, and Preset for visualization, forming a unified cloud-based stock price analytics solution.

III System Design

The system architecture follows a layered design pattern — **RAW → STAGING → INTERMEDIATE → ANALYTICS** — orchestrated by three Apache Airflow DAGs that run daily in sequence. The first DAG, YfinanceToSnowflake_ETL, extracts and loads daily OHLCV data for selected tickers (**AAPL**, **MSFT**, **AMZN**) into RAW.STOCK_PRICES.

The second DAG, ML_Forecast_DAG, builds a Snowflake ML Forecast model on the raw data and stores 7-day predictions

in MODEL.FORECASTS. Finally, the built_elt_dbt_dag orchestrates dbt transformations to generate technical indicators (**Moving Averages**, **RSI**, **Momentum**, and **Daily Returns**) across the **STAGING**, **INTERMEDIATE**, and **ANALYTICS** layers.

The resulting ANALYTICS.FINAL_STOCK_PRICES_TABLE table serves as the data source for the Preset dashboard, as illustrated in Figure 1.

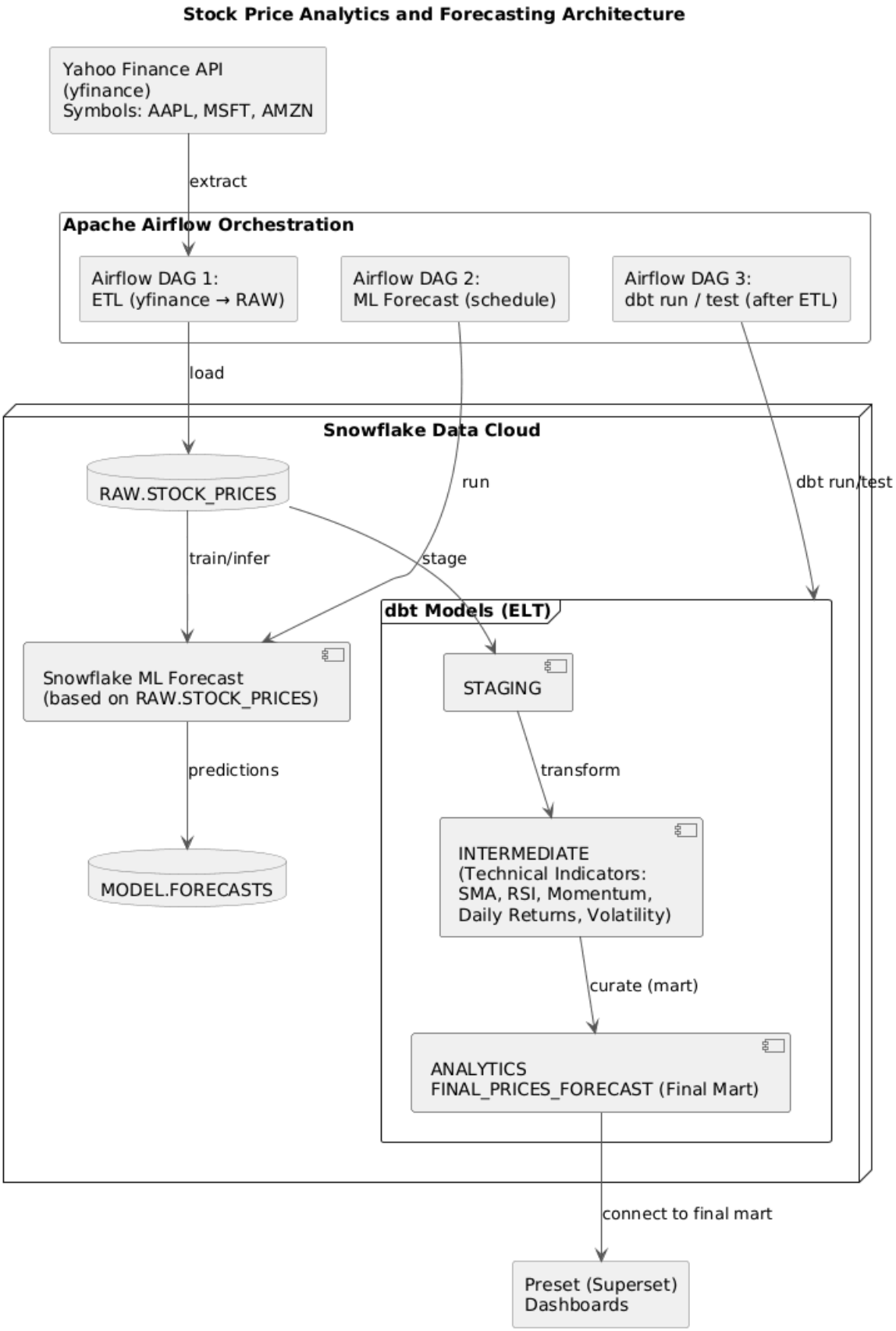


Fig. 1: System architecture of the end-to-end stock price analytics pipeline showing ETL, ELT, and ML forecasting stages

orchestrated by Apache Airflow and visualized through Preset.

IV Implementation

The implementation phase focuses on developing automated data pipelines using **Apache Airflow**, **Snowflake**, **dbt**, and **Preset**. Airflow orchestrates ETL, ELT, and ML Forecasting workflows; Snowflake provides secure data storage and machine learning; dbt handles data transformation and modeling; and Preset visualizes analytical results. All workflows are scheduled daily to ensure data freshness and automation.

IV-A ETL with Apache Airflow (Lab 1)

The primary ETL DAG (**YfinanceToSnowflake_ETL**) executes daily at **03:30 AM** and extracts the last 180+ days of OHLCV data for **AAPL, MSFT, and AMZN** using the **yfinance** library. Airflow Variables centrally manage the ticker list and table parameters for easy extension without code changes. Data is temporarily saved as CSV, then loaded idempotently into the raw table **RAW.STOCK_PRICES** using a **DELETE + COPY INTO** pattern wrapped in explicit SQL transactions **(BEGIN/COMMIT/ROLLBACK)** and Python try/except blocks. This guarantees exactly-once loading even on retries or overlapping runs, satisfying the idempotency requirement of Lab 1.

IV-B Snowflake ML Forecasting (Lab 1)

The second DAG, **ML_Forecast_DAG**, automates model training and prediction using Snowflake's native ML Forecast capability. It creates a training view from **RAW.STOCK_PRICES**, trains a forecasting model per symbol, and generates **7-day predictions stored in MODEL.FORECASTS**. The process runs within transactional blocks to ensure reliability and uses Airflow Variables and Connections for configuration and secure access. Scheduled at **04:00 AM**, this DAG ensures that new forecasts are produced daily for integration with analytics and visualization layers.

IV-C ELT with dbt (Lab 2)

The third DAG, built_elt_dbt_dag, integrates dbt with Airflow using the BashOperator to automate the ELT workflow. It runs the dbt models as part of an Airflow DAG scheduled after ETL, performing specific calculations (e.g., Moving Averages, RSI) in dbt. The dbt project codes implement models, tests, and snapshot, with screenshots of dbt commands for run/test/snapshot.This DAG is **scheduled at 03:45 AM**, immediately after the ETL pipeline finishes, ensuring that all transformations run on the most recent loaded data.

1) Staging Layer

The staging_stock_prices.sql model, materialized as a **view**, cleans and standardizes raw data from RAW.STOCK_PRICES. It converts columns to proper data types (DATE, FLOAT, NUMBER), uppercases ticker symbols for consistency, and prepares the dataset for downstream models. The accompanying staging_stock_prices.yml file defines metadata and data quality tests such as not_null and unique_combination_of_columns to ensure data integrity, supported by the dbt-utils package.

2) Intermediate Layer

All intermediate transformations are materialized as **dbt views** within the INTERMEDIATE schema. These models enrich the staged data with derived financial indicators used for performance, trend, and volatility analysis:

- **int_daily_returns.sql** – Calculates the percentage change in closing price using the lag() function, generating the metric daily_return_pct to represent day-over-day returns.

  Formula: Daily Return (%) = [(Current Close - Previous Close) / Previous Close] × 100

- **int_roc_momentum.sql** – Computes 10-day Rate of Change (roc_10) and Momentum (momentum_10) to capture the speed and direction of price movement trends.

  Formulas:

  Rate of Change (ROC, %) = [(Current Close - Close 10 days ago) / Close 10 days ago] × 100

  Momentum = Current Close - Close 10 days ago

- **int_sma.sql** – Derives 20-day and 50-day Simple Moving Averages (sma_20, sma_50)to highlight short- and medium-term price trends, where SMA crossovers indicate bullish or bearish signals.

  Formula: SMA_n = (Sum of Closing Prices over last n days) / n

(e.g., SMA_20 = (Close_today + Close_yesterday + ... + Close_19 days ago) / 20)
Explanation: Averages closing prices over a window to smooth trends; 20-day for short-term, 50-day for medium-term.Crossovers (e.g., SMA_20 > SMA_50 = bullish)
signal buy/sell opportunities.

- **int_volatility.sql** – Calculates 20-day rolling standard deviation of daily returns (vol_20d) to measure market volatility and price risk.
  Formula: Volatility_n = Standard Deviation of Daily Returns over last n days
  (Where Daily Return = [(Close_i - Close_{i-1}) / Close_{i-1}] for each day i in the window)
  Explanation:
  Quantifies price fluctuation risk; higher values indicate instability. Computed as the rolling STDDEV of daily returns (reference daily_return_pct from the previous model).

Each model includes not_null and relationships tests to ensure valid keys and referential consistency with the staging layer, defined in intermediate.yml. Metric columns are excluded from not_null testing since rolling calculations may produce NULL values in initial windows.

3) Marts Layer

The final_stock_price_table.sql model, materialized as a **table** in the ANALYTICS schema, combines all intermediate indicators–daily returns, moving averages, rate of change, momentum, and volatility—into a unified dataset for BI visualization. It creates a unique_key for each (symbol, date) pair to ensure record uniqueness. The corresponding schema.yml file includes not_null, unique, and accepted_range tests to verify completeness, accuracy, and metric stability before data is visualized in **Preset (Superset)** dashboards.

4) Snapshots

A dbt snapshot, snapshot_stock_prices, is implemented to capture historical changes in stock data over time. It uses the **check strategy** to monitor changes in key columns (open, high, low, close, volume) and assigns a composite key based on symbol and date. The snapshot is stored in the SNAPSHOTS schema, enabling version control and point-in-time analysis of stock prices.

IV-D Preset Dashboard Visualization (Lab 2)

The Preset dashboard connects to the final mart table **ANALYTICS.FINAL_STOCK_PRICES_TABLE.sql**, displaying key metrics like moving averages, RSI, price momentum, and volatility. It includes filters for ticker (AAPL, MSFT, AMZN) and date range. The dashboard's purpose is to provide interactive stock trend analysis, with usage via selecting filters to update all charts in real-time.

IV-E Testing and Idempotency

Idempotency is ensured in the ETL DAG via MERGE-equivalent DELETE + COPY INTO within SQL transactions (BEGIN/COMMIT/ROLLBACK) and Python try/except. dbt tests validate not_null, unique, and relationships across layers. Screenshots show successful dbt test runs with no failures.

V Results and Screenshots

V-A DAG1

YfinanceToSnowflake_ETL Overview

This Airflow DAG extracts the last 180 days of OHLCV stock data for tickers (AAPL, MSFT, AMZN) from Yahoo Finance (yfinance) and loads it into Snowflake.
It runs daily at 03:30 AM UTC and finishes before the dbt ELT pipeline.

Workflow:

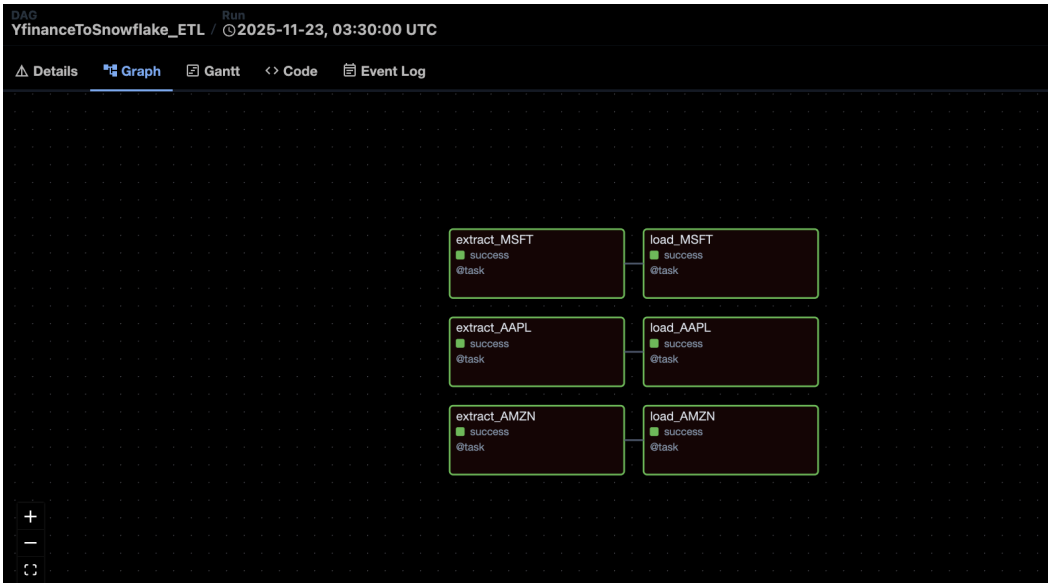
- Downloads data from yfinance for each symbol based on the Airflow logical date.
- Saves data as CSV in /tmp/, uploads to a temporary Snowflake stage, and loads into USER_DB_FERRET.RAW.STOCK_PRICES using COPY INTO.
- Deletes overlapping rows (same symbol/date) for idempotency and wraps all operations in SQL transactions (BEGIN, COMMIT, ROLLBACK).

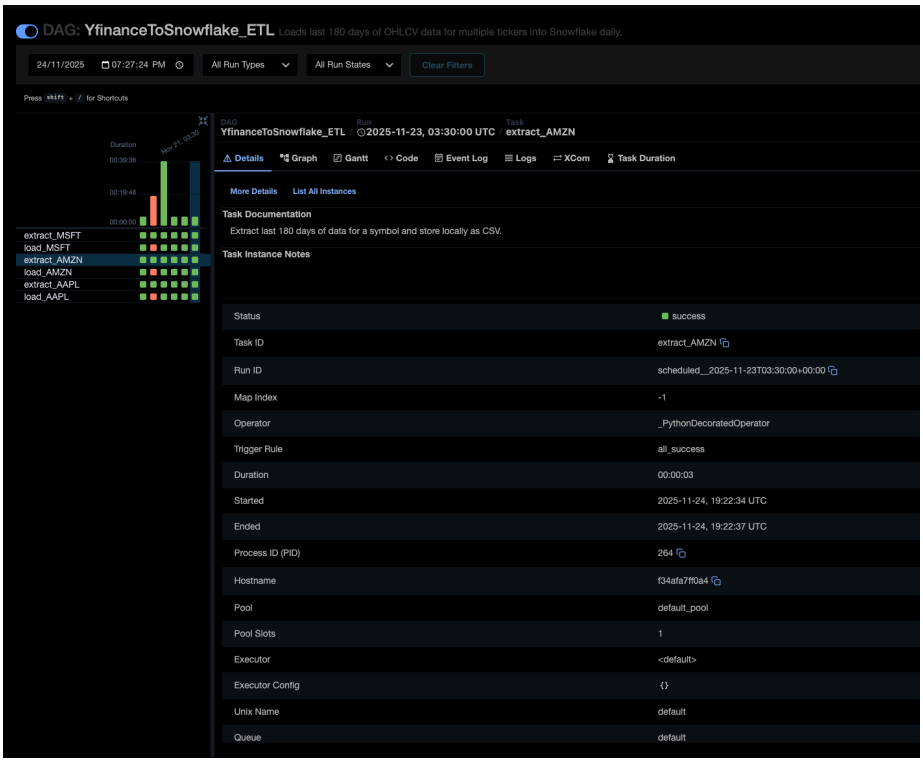
Configuration:

- Airflow Variables: snowflake_db, snowflake_schema, snowflake_table, stock_symbols.
- Connection: my_snowflake_conn.
- Dynamically maps extract–load tasks for each symbol.

Output:
Creates a clean, deduplicated stock price table in RAW.STOCK_PRICES, ready for dbt transformations, ML forecasting, and dashboard visualization.

DAG1 result:





INPUT TABLE

Table definition

```
create or replace TABLE USER_DB_FERRET.RAW.STOCK_PRICES (
    DATE DATE,
    OPEN FLOAT,
    CLOSE FLOAT,
    HIGH FLOAT,
    LOW FLOAT,
    VOLUME NUMBER(38,0),
    SYMBOL VARCHAR(16777216)
);
```

Show less

Output of RAW.STOCK_PRICES:

Table | Chart | 378 rows | 755ms

	DATE	OPEN	CLOSE	HIGH	LOW	VOLUME	SYMBOL
	22/05... 19/11...	193.66... 555.22...	195.27... 542.07...	197.57... 555.45...	193.46... 540.77...	1184... 1663...	AAPL 33.3% AMZN 33.3% +1 more
1	2025-05-23	193.669998169	195.270004272	197.699996948	193.460006714	78432900	AAPL
2	2025-05-23	449.980010986	450.179992676	453.690002441	448.910003662	16883500	MSFT
3	2025-05-23	198.899993896	200.990005493	202.369995117	197.850006104	33393500	AMZN
4	2025-05-27	198.300003052	200.210006714	200.740005493	197.429992676	56288500	AAPL
5	2025-05-28	200.589996338	200.419998169	202.729995728	199.899993896	45339700	AAPL
6	2025-05-29	203.580001831	199.949996948	203.809997559	198.509994507	51396800	AAPL
7	2025-05-30	199.369995117	200.850006104	201.960006714	196.779998779	70819900	AAPL
8	2025-06-02	200.279998779	201.699996948	202.130004883	200.119995117	35423300	AAPL
9	2025-06-03	201.350006104	203.270004272	203.770004272	200.960006714	46381600	AAPL
10	2025-06-04	202.910003662	202.820007324	206.240005493	202.100006104	43604000	AAPL
11	2025-06-05	203.5	200.630004883	204.75	200.149993896	55126100	AAPL

V-B DAG2

ML_Forecast_DAG Overview

This Airflow pipeline automates **daily training and prediction** using **Snowflake's native ML Forecast** model.
It runs every day at **04:00 AM**.

Workflow Summary

- **Input:** USER_DB_FERRET.RAW.STOCK_PRICES — actual stock data.
- **Training View:** USER_DB_FERRET.ADHOC.MARKET_DATA_VIEW — built from DATE, CLOSE, and SYMBOL.
- **Model & Outputs:** USER_DB_FERRET.MODEL.FORECASTS — stores 7-day forecast results.
- **Final Table:** USER_DB_FERRET.ANALYTICS.FINAL_STOCK_PRICES_FORECAST — combines actual and forecast records for BI dashboards.

Tasks

- train task:
  Creates the training view, builds or refreshes the SNOWFLAKE.ML.FORECAST object, logs evaluation metrics, and executes all SQL inside a BEGIN/COMMIT/ROLLBACK transaction.
- predict task:
  Calls the forecast function to generate 7-day predictions, saves results to MODEL.FORECASTS, and merges actuals and forecasts into the final analytics table.

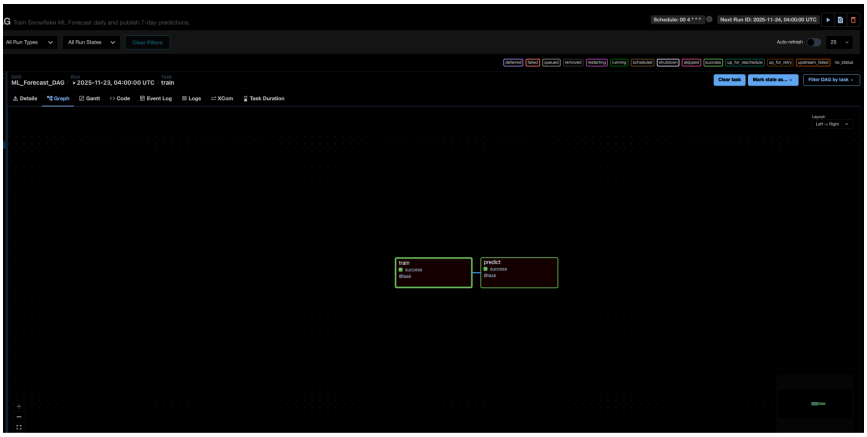
Configuration & Output

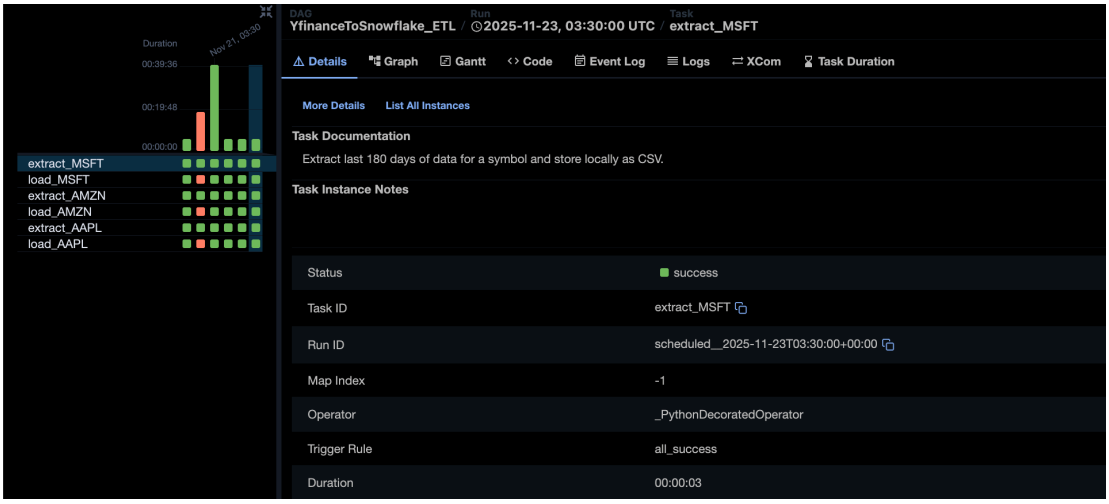
The DAG reads configuration parameters from Airflow Variables (etl_table, train_view, forecast_table, final_table, forecast_fn)
and connects securely to Snowflake via the Airflow Snowflake Connection.

Upon completion, it produces:

- Up-to-date 7-day forecasts in MODEL.FORECASTS.
- A BI-ready combined dataset in ANALYTICS.FINAL_STOCK_PRICES_FORECAST.

DAG2 Result:





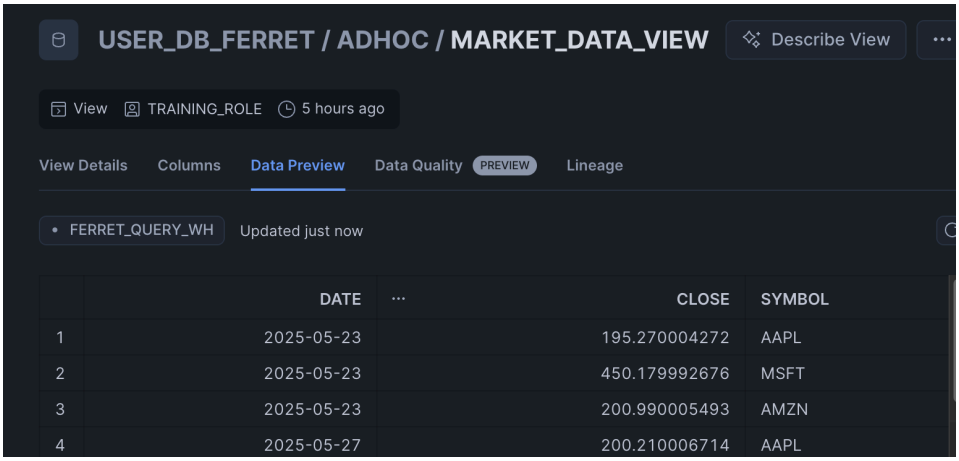
ADHOC :

INPUT TABLE

```
create or replace view USER_DB_FERRET.ADHOC.MARKET_DATA_VIEW(
    DATE,
    CLOSE,
    SYMBOL
) as
        SELECT DATE, CLOSE, SYMBOL
        FROM USER_DB_FERRET.RAW.STOCK_PRICES;
```

Show less

OUTPUT TABLE:



	DATE	CLOSE	SYMBOL
1	2025-05-23	195.270004272	AAPL
2	2025-05-23	450.179992676	MSFT
3	2025-05-23	200.990005493	AMZN
4	2025-05-27	200.210006714	AAPL

MODEL :

INPUT TABLE

Table definition

```
create or replace TABLE USER_DB_FERRET.MODEL.FORECASTS (
    SERIES VARIANT,
    TS TIMESTAMP_NTZ(9),
    FORECAST FLOAT,
    LOWER_BOUND FLOAT,
    UPPER_BOUND FLOAT
);
```

Show less

OUTPUT TABLE:

USER_DB_FERRET / MODEL / FORECASTS Describe Table ··· Load Data

Table TRAINING_ROLE 5 hours ago 21 3.0KB

Table Details Columns Data Preview Copy History Data Quality PREVIEW Lineage

FERRET_QUERY_WH 21 Rows • Updated just now

	SERIES	···	TS	FORECAST	LOWER_BOUND
1	"MSFT"		2025-11-20T00:00:00.000+0000	487.601989746	477.822625987
2	"MSFT"		2025-11-21T00:00:00.000+0000	487.693939209	472.156577555
3	"MSFT"		2025-11-24T00:00:00.000+0000	487.782226562	468.605989838
4	"MSFT"		2025-11-25T00:00:00.000+0000	487.866973877	466.229115804
5	"MSFT"		2025-11-26T00:00:00.000+0000	487.948364258	464.5465566
6	"MSFT"		2025-11-27T00:00:00.000+0000	488.026519775	464.210948944
7	"MSFT"		2025-11-28T00:00:00.000+0000	488.101501465	460.388098399
8	"AAPL"		2025-11-20T00:00:00.000+0000	269.59178173	267.546964021
9	"AAPL"		2025-11-21T00:00:00.000+0000	269.684438002	267.540302349
10	"AAPL"		2025-11-24T00:00:00.000+0000	269.471432925	267.232373803
11	"AAPL"		2025-11-25T00:00:00.000+0000	271.067523589	268.737407766
12	"AAPL"		2025-11-26T00:00:00.000+0000	269.157720061	266.739970738
13	"AAPL"		2025-11-27T00:00:00.000+0000	270.950309071	268.447988645
14	"AAPL"		2025-11-28T00:00:00.000+0000	271.176696017	268.592590291
15	"AMZN"		2025-11-20T00:00:00.000+0000	223.081323663	215.241836016

ANALYTICS :

INPUT TABLE

```sql
create or replace TABLE
USER_DB_FERRET.ANALYTICS.FINAL_PRICES_FORECAST (
    SYMBOL VARCHAR(16777216),
    DATE DATE,
    RECORD_TYPE VARCHAR(8),
    PRICE FLOAT,
    LOWER_BOUND FLOAT,
    UPPER_BOUND FLOAT
);
```

OUTPUT TABLE (ACTUAL):

	SYMBOL	DATE	RECORD_TYPE	PRICE	LOWER_BOUND	UPPER_BOUND
	AAPL 33.3% AMZN 33.3% +1 more	22/05/... 27/11/...	ACTUAL 94.7% FORECAST 5.3%	195.270... 542.070...	208.7601... 477.8226...	230.9208... 515.4478...
1	AAPL	2025-05-23	ACTUAL	195.270004272	null	null
2	MSFT	2025-05-23	ACTUAL	450.179992676	null	null
3	AMZN	2025-05-23	ACTUAL	200.990005493	null	null

OUTPUT TABLE(FORECAST):

Table Chart 21 rows 34ms

	SYMBOL	DATE	RECORD_TYPE	PRICE	LOWER_BOUND	UPPER_BOUND
	AAPL 33.3% AMZN 33.3% +1 more	19/11/... 27/11/...	FORECAST 100.0%	223.081... 488.101...	208.7601... 477.8226...	230.9208... 515.4478...
6	MSFT	2025-11-27	FORECAST	488.026519775	464.210948944	510.570877584
7	MSFT	2025-11-28	FORECAST	488.101501465	460.388098399	515.447821045
8	AAPL	2025-11-20	FORECAST	269.59178173	267.546964021	271.636615949
9	AAPL	2025-11-21	FORECAST	269.684438002	267.540302349	271.828598669
10	AAPL	2025-11-24	FORECAST	269.471432925	267.232373803	271.710511558
11	AAPL	2025-11-25	FORECAST	271.067523589	268.737407766	273.397659925
12	AAPL	2025-11-26	FORECAST	269.157720061	266.739970738	271.575483391
13	AAPL	2025-11-27	FORECAST	270.950309071	268.447988645	273.452627997
14	AAPL	2025-11-28	FORECAST	271.176696017	268.592590291	273.760824756
15	AMZN	2025-11-20	FORECAST	223.081323663	215.241836016	230.920811309

V-C DAG3

BuildELT_dbt Overview

The **BuildELT_dbt** DAG automates the dbt workflow in **Snowflake** using the **BashOperator**. It runs sequentially after the ETL DAG everyday at **3:45 AM** and executes:

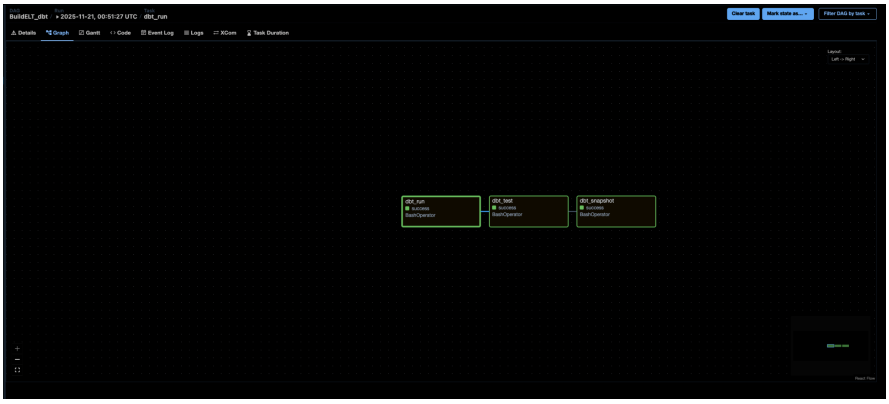
1. **dbt run** – builds staging, intermediate, and final models.
2. **dbt test** – validates data using not_null, unique, and relationships tests.
3. **dbt snapshot** – records historical changes for version control.

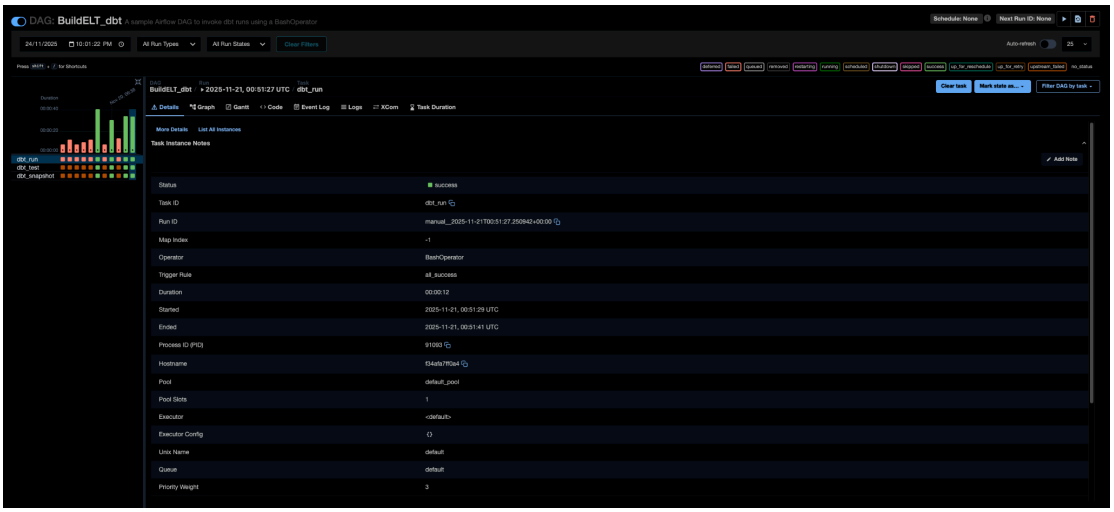
Workflow Summary:

- The **STAGING** layer cleans and standardizes raw stock data from RAW.STOCK_PRICES.
- The **INTERMEDIATE** layer (stored in INTERMEDIATE schema) computes four indicators: **Daily Returns**, **ROC & Momentum**, **SMA (20 & 50-day)**, and **Volatility**.
- The **ANALYTICS** layer merges all metrics into a single table, ANALYTICS.FINAL_STOCK_PRICE_TABLE, used for dashboard visualization in Preset.

Output:
The DAG ensures that transformed data is always fresh, tested, and analytics-ready for visualization and reporting.

DAG 3 RESULT:





A. STAGING

INPUT

dbt > lab2_stock_analytics > models > Staging > staging_stock_prices.sql

```sql
-- models/Staging/staging_stock_prices.sql
{{ config(materialized='view') }}

with sp as (
  select
    date::date   as date,
    open::float  as open,
    high::float  as high,
    low::float   as low,
    close::float as close,
    volume::number as volume,
    upper(symbol) as symbol
  from {{ source('raw', 'STOCK_PRICES') }}
)

select * from sp
```

dbt > lab2_stock_analytics > models > Staging > staging.yml

```yaml
version: 2

models:
  - name: staging_stock_prices
    description: "Cleaned and standardized stock price data from RAW.STOCK_PRICES."
    columns:
      - name: symbol
        description: "Ticker symbol for the stock."
        tests:
          - not_null
      - name: date
        description: "Trading date."
        tests:
          - not_null
      - name: close
        description: "Daily closing price."
        tests:
          - not_null
    tests:
      - dbt_utils.unique_combination_of_columns:
          arguments:
            combination_of_columns: [symbol, date]
```

Staging.staging_stock_prices output

Table | Chart | 378 rows | 281ms

	DATE	OPEN	HIGH	LOW	CLOSE	VOLUME	SYMBOL
	22/05... 19/11...	193.66... 555.22...	197.57... 555.45...	193.46... 540.77...	195.27... 542.07...	1184... 1663...	AAPL 33.3% AMZN 33.3% +1 more
1	2025-11-20	227.050003052	227.410003662	216.740005493	217.13999939	50060800	AMZN
2	2025-11-20	270.829986572	275.429992676	265.920013428	266.25	45693900	AAPL
3	2025-11-20	492.709991455	493.570007324	475.5	478.429992676	26688100	MSFT

B. INTERMEDIATE

1. DAILY RETURNS TRANSFORMATION CODE

INPUT:

dbt > lab2_stock_analytics > models > Intermediate > int_daily_returns.sql

```sql
-- Model: int_daily_returns
-- Purpose: Calculate the daily percentage change in closing price
--          for each stock symbol using lag() over date order.
-- Layer: Intermediate (INTERMEDIATE schema)

{{ config(materialized='view') }}

with base as (
  select
    symbol,
    date,
    close,
    lag(close) over (partition by symbol order by date) as prev_close
  from {{ ref('staging_stock_prices') }}
)

select
  symbol,
  date,
  case
    when prev_close is null or prev_close = 0 then null
    else ((close - prev_close) / prev_close) * 100.0
  end as daily_return_pct
from base
```

OUTPUT:

Table | Chart | 15 rows | 351ms

	SYMBOL	DATE	DAILY_RETURN_PCT
	AAPL 33.3% AMZN 33.3% +1 more	17/11/2025 – 23/11/2025	-4.431653833 – 2.532963105
1	AAPL	2025-11-24	1.631744577
2	AAPL	2025-11-21	1.96807145
3	AAPL	2025-11-20	-0.8601420836
4	AAPL	2025-11-19	0.4187836924
5	AAPL	2025-11-18	-0.007473646269
6	AMZN	2025-11-24	2.532963105
7	AMZN	2025-11-21	1.634891343
8	AMZN	2025-11-20	-2.492255149
9	AMZN	2025-11-19	0.06290693675
10	AMZN	2025-11-18	-4.431653833
11	MSFT	2025-11-24	0.3982048848
12	MSFT	2025-11-21	-1.318896736
13	MSFT	2025-11-20	-1.783955191
14	MSFT	2025-11-19	-1.350779342
15	MSFT	2025-11-18	-2.699557026

2. ROC MOMENTUM TRANSFORMATION CODE

INPUT:

dbt > lab2_stock_analytics > models > Intermediate > int_roc_momentum.sql

```sql
-- Model: int_roc_momentum
-- Purpose: Calculate 10-day Rate of Change (ROC) and Momentum
--          to measure the speed and direction of price movements.
-- Layer: Intermediate (INTERMEDIATE schema)


{{ config(materialized='view') }}

with base as (
  select
    symbol,
    date,
    close,
    lag(close, 10) over (partition by symbol order by date) as close_lag_10
  from {{ ref('staging_stock_prices') }}
)

select
  symbol,
  date,
  case
    when close_lag_10 is null or close_lag_10 = 0 then null
    else ((close / close_lag_10) - 1) * 100.0
  end as roc_10,
  case
    when close_lag_10 is null then null
    else close - close_lag_10
  end as momentum_10
from base

```

OUTPUT:

Table | Chart | 15 rows | 286ms

	SYMBOL	DATE	ROC_10	MOMENTUM_10
	AAPL 33.3% AMZN 33.3% +1 more	17/11/2025 – 23/11/2025	-10.995201776 – 2.408796692	-32 – 6.490020752
1	AAPL	2025-11-24	2.408796692	6.490020752
2	AAPL	2025-11-21	1.124888814	3.019989014
3	AAPL	2025-11-20	-1.304811194	-3.519989014
4	AAPL	2025-11-19	-0.5848882077	-1.58001709
5	AAPL	2025-11-18	-0.9628225527	-2.600006104
6	AMZN	2025-11-24	-8.904990202	-22.119995117
7	AMZN	2025-11-21	-9.705004241	-23.720001221
8	AMZN	2025-11-20	-10.656679811	-25.899993896
9	AMZN	2025-11-19	-10.995201776	-27.509994507
10	AMZN	2025-11-18	-10.737206596	-26.770004272
11	MSFT	2025-11-24	-6.324110672	-32
12	MSFT	2025-11-21	-4.971621884	-24.700012207
13	MSFT	2025-11-20	-3.7557862	-18.670013428
14	MSFT	2025-11-19	-3.951417383	-20.040008545
15	MSFT	2025-11-18	-3.993546529	-20.540008545

3. SMA TRANSFORMATION CODE:

INPUT:

dbt > lab2_stock_analytics > models > Intermediate > int_sma.sql

```sql
-- Model: int_sma
-- Purpose: Compute simple moving averages (SMA-20 and SMA-50)
--          to identify short- and medium-term stock trends.
-- Layer: Intermediate (INTERMEDIATE schema)


{{ config(materialized='view') }}

select
  symbol,
  date,
  avg(close) over (
    partition by symbol
    order by date
    rows between 19 preceding and current row
  ) as sma_20,
  avg(close) over (
    partition by symbol
    order by date
    rows between 49 preceding and current row
  ) as sma_50
from {{ ref('staging_stock_prices') }}
```

OUTPUT:

Table | Chart | 15 rows | 422ms

	SYMBOL	DATE	SMA_20	SMA_50
	AAPL 33.3% AMZN 33.3% +1 more	17/11/2025 – 23/11/2025	236.189500427 – 514.821502686	227.66440033 – 514.352598877
1	AAPL	2025-11-24	270.428500366	260.272399292
2	AAPL	2025-11-21	270.072999573	259.487998962
3	AAPL	2025-11-20	269.639500427	258.739599304
4	AAPL	2025-11-19	269.305999756	258.01519928
5	AAPL	2025-11-18	268.800500488	257.179799194
6	AMZN	2025-11-24	236.189500427	227.66440033
7	AMZN	2025-11-21	236.224000549	227.767400208
8	AMZN	2025-11-20	236.400000763	227.916600037
9	AMZN	2025-11-19	236.59750061	228.172799988
10	AMZN	2025-11-18	236.360500336	228.325599976
11	MSFT	2025-11-24	505.593502808	512.053198853
12	MSFT	2025-11-21	508.469503784	512.88039856
13	MSFT	2025-11-20	511.044003296	513.635998535
14	MSFT	2025-11-19	513.15050354	514.087598877
15	MSFT	2025-11-18	514.821502686	514.352598877

4. VOLATILITY TRANSFORMATION CODE

INPUT

dbt > lab2_stock_analytics > models > Intermediate > int_volatility.sql

```sql

-- Model: int_volatility
-- Purpose: Calculate 20-day rolling volatility (standard deviation
--          of daily returns) as a measure of price fluctuation.
-- Layer: Intermediate (INTERMEDIATE schema)

{{ config(materialized='view') }}

-- Volatility as rolling 20-day stddev of daily returns
with dr as (
  select
    symbol,
    date,
    daily_return_pct
  from {{ ref('int_daily_returns') }}
)

select
  symbol,
  date,
  stddev_samp(daily_return_pct) over (
    partition by symbol
    order by date
    rows between 19 preceding and current row
  ) as vol_20d
from dr

```

OUTPUT:

Table | Chart | 15 rows | 380ms

	SYMBOL	DATE	VOL_20D
	AAPL 33.3% AMZN 33.3% +1 more	17/11/2025 – 23/11/2025	0.9227999965 – 3.099674882
1	AAPL	2025-11-24	0.9435980077
2	AAPL	2025-11-21	1.006647746
3	AAPL	2025-11-20	0.950149875
4	AAPL	2025-11-19	0.9227999965
5	AAPL	2025-11-18	1.00794693
6	AMZN	2025-11-24	3.099674882
7	AMZN	2025-11-21	3.057712453
8	AMZN	2025-11-20	3.051682522
9	AMZN	2025-11-19	3.012286139
10	AMZN	2025-11-18	3.044963984
11	MSFT	2025-11-24	1.399916528
12	MSFT	2025-11-21	1.460838994
13	MSFT	2025-11-20	1.467135427
14	MSFT	2025-11-19	1.433139454
15	MSFT	2025-11-18	1.424466208

Intermediate .yml

```yaml
# Layer: INTERMEDIATE
# Purpose: Test integrity of intermediate analytical views
#          that compute technical indicators (returns, SMA, ROC, volatility).
# Tests:
#   • not_null - Ensures each intermediate table has valid symbol/date keys.
#   • relationships - Confirms symbol/date values align with staging data.
# Notes:
#   Metric columns are not tested for not_null since rolling functions
#   naturally produce NULLs during initial windows.
```

```yaml
# Outcome:
#   Verifies integrity of intermediate layer and linkage to staging.

version: 2

models:
 - name: int_daily_returns
   columns:
     - name: symbol
       tests: [not_null]
     - name: date
       tests:
         - not_null
         - relationships:
             to: ref('staging_stock_prices')
             field: date

 - name: int_sma
   columns:
     - name: symbol
       tests: [not_null]
     - name: date
       tests:
         - not_null
         - relationships:
             to: ref('staging_stock_prices')
             field: date

 - name: int_roc_momentum
   columns:
     - name: symbol
       tests: [not_null]
     - name: date
       tests:
         - not_null
         - relationships:
             to: ref('staging_stock_prices')
             field: date

 - name: int_volatility
   columns:
     - name: symbol
       tests: [not_null]
```

```yaml
      - name: date
        tests:
          - not_null
          - relationships:
              to: ref('staging_stock_prices')
              field: date
```

C. MARTS

The MARTS layer is conceptually represented by the ANALYTICS schema, where all final analytical tables are stored for BI visualization.

INPUT:

dbt > lab2_stock_analytics > models > Marts > final_stock_price_table.sql

```sql
-- Model: fct_technical_indicators
-- Purpose: Combine all key technical indicators (daily returns,
--          moving averages, ROC, momentum, volatility) into a
--          single analytical table for BI dashboards.
-- Layer: Mart (ANALYTICS schema)

{{ config(materialized='table') }}

with
dr  as (select * from {{ ref('int_daily_returns') }}),
sma as (select * from {{ ref('int_sma') }}),
roc as (select * from {{ ref('int_roc_momentum') }}),
vol as (select * from {{ ref('int_volatility') }})

select
  coalesce(dr.symbol, sma.symbol, roc.symbol, vol.symbol) as symbol,
  coalesce(dr.date,   sma.date,   roc.date,   vol.date)   as date,
  dr.daily_return_pct,
  sma.sma_20, sma.sma_50,
  roc.roc_10, roc.momentum_10,
  vol.vol_20d,
  {{ dbt_utils.generate_surrogate_key(['coalesce(dr.symbol, sma.symbol, roc.symbol, vol.symbo
                                       'coalesce(dr.date,   sma.date,   roc.date,   vol.date)
from dr
full outer join sma using (symbol, date)
full outer join roc using (symbol, date)
full outer join vol using (symbol, date)
```

.

OUTPUT TABLE

Table | Chart — 15 rows — 224ms

	SYMBOL	DATE	DAILY_RETURN_PCT	SMA_20	SMA_50	ROC_10	MOMENTUM_10	VOL_20D	UNIQUE_KEY
	AAPL 33.3% AMZN 33.3% +1 more	12/11/... 18/11/...	-4.4316538... 1.368988856	234.432... 516.650...	228.172... 514.444...	-10.995... 6.60504...	-27.5099... 14.72000...	0.92279... 3.04496...	2d1e27a59aa89a72f8ca6aa0fb8... 6.7% 52f4a13ec81a6fbdc9b43117a7b... 6.7% +13 more
1	MSFT	2025-11-19	-1.350779342	513.15050354	514.087598877	-3.951417383	-20.040008545	1.433139454	ee828c84d06fa402e91aeb1e84974e41
2	MSFT	2025-11-18	-2.699557026	514.821502686	514.352598877	-3.993546529	-20.540008545	1.424466208	d1867146e29e304570a982dab8773421
3	MSFT	2025-11-17	-0.527265373	516.015000916	514.444998779	-1.845161504	-9.540039062	1.301399881	fd4871fe1bd8e9b1a7cfc6d7c519cdc4
4	MSFT	2025-11-14	1.368988856	516.480000305	514.259199219	-1.473514401	-7.630004883	1.306179314	68925712b91c62868d234b1f1565a14b
5	MSFT	2025-11-13	-1.53578391	516.650001526	513.955599365	-4.273813299	-22.470001221	1.268907443	f463d74863f56c0351bc780c5081a5ec
6	AMZN	2025-11-19	0.06290693675	236.59750061	228.172799988	-10.995201776	-27.509994507	3.012286139	2d1e27a59aa89a72f8ca6aa0fb830677
7	AMZN	2025-11-18	-4.431653833	236.360500336	228.325599976	-10.737206596	-26.770004272	3.044963984	f9a02aafdfcfa61c3347bc73ef0ce2cc
8	AMZN	2025-11-17	-0.7754941861	236.334500122	228.639400024	-8.31889956	-21.130004883	2.900881989	52f4a13ec81a6fbdc9b43117a7bd5382
9	AMZN	2025-11-14	-1.216432093	235.515000153	228.698800049	-3.902218791	-9.529998779	2.898952054	66aa20d1aea45359d27de22a254ab316
10	AMZN	2025-11-13	-2.710890745	234.432499695	228.651600037	6.605044055	14.720001221	2.884152722	8d4e23cd4c7372ff6ba432bcdbfd38f0
11	AAPL	2025-11-19	0.4187836924	269.305999756	258.01519928	-0.5848882077	-1.58001709	0.9227999965	59895abb7a866eaab7c0d5efaa1ed35e
12	AAPL	2025-11-18	-0.007473646269	268.800500488	257.179799194	-0.9628225527	-2.600006104	1.00794693	a2de620f27a92a197143b6567d10076e
13	AAPL	2025-11-17	-1.817118366	268.566999817	256.517999268	-0.5909668872	-1.589996338	1.0079376	a9f0c9a08ff75872052cb25de62d1596
14	AAPL	2025-11-14	-0.1978415537	268.305999756	255.926399536	0.7545247556	2.040008545	1.229123165	5c1ff4554cc667eafb63e5fabdbeb3cb
15	AAPL	2025-11-13	-0.1901448098	267.299999237	255.271999512	0.5711195083	1.550018311	1.268495873	7a77c8fb4b4ca0a5a42a6a02966c33e3

Marts.yml

```yaml
# Layer: MARTS (FINAL)
# Purpose: Validate the final analytical table that aggregates all
#          technical indicators for BI dashboards and forecasting.
# Tests:
#   • not_null – Ensures key fields (symbol, date, unique_key) are populated.
#   • unique – Confirms there are no duplicate symbol/date records.
#   • relationships – Validates symbols align with staging layer.
#   • accepted_range – Checks for reasonable metric ranges (optional).

# Outcome:
#   Ensures final mart data is accurate, complete, and suitable for visualization.

version: 2

models:
 - name: final_stock_price_table
   description: "Final mart combining technical indicators per symbol/date."
   columns:
     # ---- Primary business key (dimension) ----
     - name: symbol
       description: "Ticker symbol."
       tests:
         - not_null
         - relationships:
             to: ref('staging_stock_prices')
             field: symbol

     # ---- Primary business key (date grain) ----
     - name: date
       description: "Trading date."
       tests:
         - not_null

     # ---- Surrogate key enforcing uniqueness at (symbol, date) ----
     - name: unique_key
       description: "Surrogate key for (symbol, date)."
       tests:
         - not_null
         - unique
```

```yaml
# ---- Metrics: use sanity ranges, not not_null  ----
- name: daily_return_pct
  description: "Day-to-day percent return ((close - lag)/lag*100)."
  tests:
    - accepted_range:
        min_value: -100
        max_value: 100

- name: sma_20
  description: "20-day moving average of close."
  tests:
    # Tip: You can omit accepted_values here; it does nothing with an empty list.
    # If you want a sanity check, prefer accepted_range with a floor of 0.
    # Example (uncomment to use):
    # - accepted_range:
    #     min_value: 0
    #     inclusive: true

- name: sma_50
  description: "50-day moving average of close."
  tests:
    - accepted_range:
        min_value: 0
        inclusive: true

- name: roc_10
  description: "10-day rate of change (%)."
  tests:
    - accepted_range:
        min_value: -100
        max_value: 100

- name: momentum_10
  description: "10-day momentum (close - close_lag_10)."
  tests: []

- name: vol_20d
  description: "Rolling 20-day stddev of daily returns."
  tests:
    - accepted_range:
        min_value: 0
        inclusive: true
```

Snapshots in dbt

Snapshots in dbt are used to capture historical versions of data so that any changes to records over time can be tracked.
In this project, a snapshot named **snapshot_stock_prices.sql** was created to record changes in daily stock price attributes such as *open*, *high*, *low*, *close*, and *volume*.

The snapshot stores its results in the **SNAPSHOTS** schema using the *check strategy*. This strategy automatically detects any changes in the specified columns and creates new rows with two system-generated timestamps — **dbt_valid_from** and **dbt_valid_to** — representing the time range during which each record was valid.

A screenshot of the snapshot configuration and its resulting table in Snowflake are shown below:

dbt Snapshot Configuration for Stock Prices:

dbt > lab2_stock_analytics > snapshots > snapshot_stock_prices.sql

```
{% snapshot snapshot_stock_prices %}

{{
  config(
    target_database='USER_DB_FERRET',
    target_schema='SNAPSHOTS',
    unique_key="symbol || '-' || to_char(date, 'YYYY-MM-DD')",
    strategy='check',
    check_cols=['open', 'high', 'low', 'close', 'volume']
  )
}}

select
  symbol,
  date,
  open,
  high,
  low,
  close,
  volume
from {{ source('raw', 'STOCK_PRICES') }}

{% endsnapshot %}
```

Snapshot Table (USER_DB_FERRET.SNAPSHOTS.SNAPSHOT_STOCK_PRICES) in Snowflake:

Below Screenshot displays the snapshot table USER_DB_FERRET.SNAPSHOTS.SNAPSHOT_STOCK_PRICES in Snowflake. Each record captures the state of stock data (Open, High, Low, Close, Volume) along with dbt-generated metadata fields — DBT_UPDATED_AT, DBT_VALID_FROM, and DBT_VALID_TO. These timestamps indicate when each record became valid and when it was superseded by a new version, allowing historical tracking of price changes over time.

Table Chart 15 rows 76ms

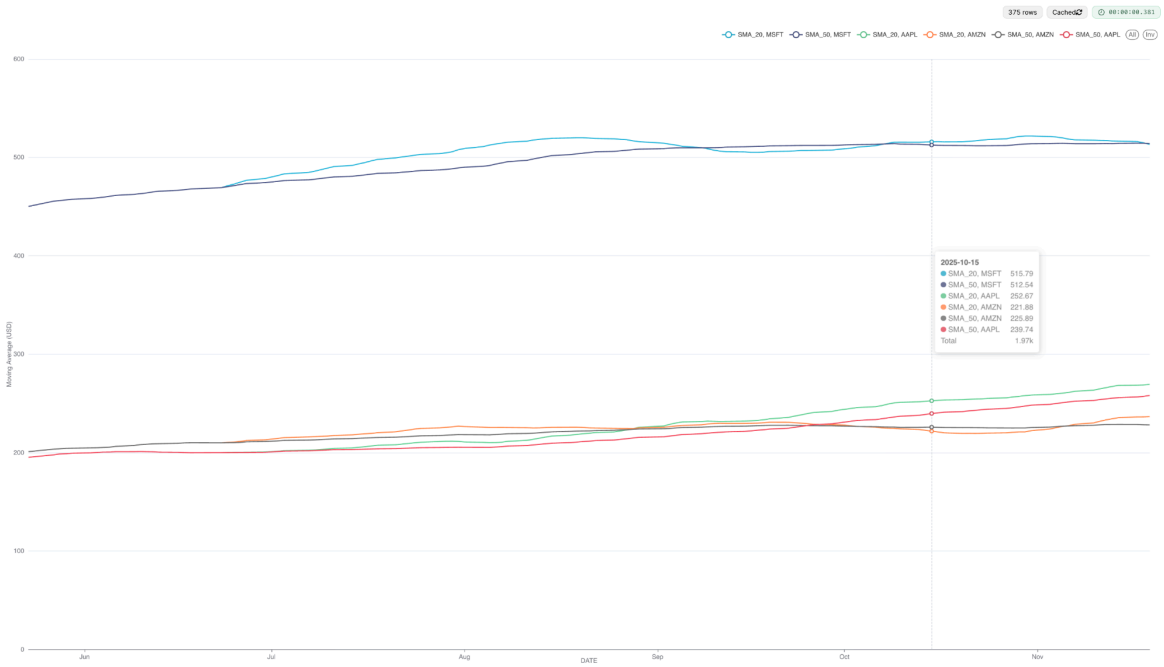
	SYMBOL	DATE	OPEN	HIGH	LOW	CLOSE	VOLUME	DBT_SCD_ID	DBT_UPDATED_AT	DBT_VALID_FROM	DBT_VALID_TO
	AAPL 33.3% AMZN 33.3% +1 more	12/11/... 18/11/...	223.735... 510.309...	223.735... 513.5	218.529... 504.910...	222.550... 510.179...	1909... 6060...	04b14abdf91b66d2fe5e9682... 6.7% 088c3d3d194eed30b8d0f93f... 6.7% +13 more	19/11/2025 19/11/2025	19/11/2025 19/11/2025	All values are null
1	MSFT	2025-11-19	490.100006104	495.187194824	482.829986572	487.119995117	22340405	b8a6ec6ca15feacc22bbd70df48f9a1d	2025-11-20 06:06:58.280	2025-11-20 06:06:58.280	null
2	MSFT	2025-11-18	495.369995117	502.980010986	486.779998779	493.790008545	33815100	14e54056d1b7429dc0c600638f36ba27	2025-11-20 06:06:58.280	2025-11-20 06:06:58.280	null
3	MSFT	2025-11-17	508.450012207	512.119995117	504.910003662	507.489990234	19092800	db134f9246679459518be92295b826a0	2025-11-20 06:06:58.280	2025-11-20 06:06:58.280	null
4	MSFT	2025-11-14	498.230010986	511.600006104	497.440002441	510.179992676	28505700	e9e7842b92502a55df70fee8122e8af7	2025-11-20 06:06:58.280	2025-11-20 06:06:58.280	null
5	MSFT	2025-11-13	510.309997559	513.5	501.290008545	503.290008545	25273100	e11f8ac7706598df688d651f47a2fe6c	2025-11-20 06:06:58.280	2025-11-20 06:06:58.280	null
6	AMZN	2025-11-19	223.73500061	223.73500061	218.529998779	222.690002441	56682003	97dc6531b00ff0155db533208350a3d2	2025-11-20 06:06:58.280	2025-11-20 06:06:58.280	null
7	AMZN	2025-11-18	228.100006104	230.199996948	222.419998169	222.550003052	60608400	b172753345e9b78ee13733d5619bd7a8	2025-11-20 06:06:58.280	2025-11-20 06:06:58.280	null
8	AMZN	2025-11-17	233.25	234.600006104	229.190002441	232.869995117	59919000	8f72f2aef046c05ae3aa02118b7f5acd	2025-11-20 06:06:58.280	2025-11-20 06:06:58.280	null
9	AMZN	2025-11-14	235.059997559	238.729995728	232.88999939	234.690002441	38956700	55ef670ad3bb85e66fec62996ecb9b7d	2025-11-20 06:06:58.280	2025-11-20 06:06:58.280	null
10	AMZN	2025-11-13	243.050003052	243.75	236.5	237.580001831	41401700	9882bc36b9075b62673369fd51c77c5c	2025-11-20 06:06:58.280	2025-11-20 06:06:58.280	null
11	AAPL	2025-11-19	265.524993896	272.209991455	265.5	268.559997559	35871303	b8512aaf310794be5166b751ed37ca98	2025-11-20 06:06:58.280	2025-11-20 06:06:58.280	null
12	AAPL	2025-11-18	269.989990234	270.709991455	265.320007324	267.440002441	45677300	04b14abdf91b66d2fe5e96822880f93b	2025-11-20 06:06:58.280	2025-11-20 06:06:58.280	null
13	AAPL	2025-11-17	268.820007324	270.489990234	265.730010986	267.459991455	45018300	088c3d3d194eed30b8d0f93fb0e2de5a	2025-11-20 06:06:58.280	2025-11-20 06:06:58.280	null
14	AAPL	2025-11-14	271.049987793	275.959991455	269.600006104	272.410003662	47431300	96f4d014959cf81c92417fbdceba8bdd	2025-11-20 06:06:58.280	2025-11-20 06:06:58.280	null
15	AAPL	2025-11-13	274.109985352	276.700012207	272.089996338	272.950012207	49602800	2801a933e0c09c256a1a95b882890f26	2025-11-20 06:06:58.280	2025-11-20 06:06:58.280	null

VI. Dashboard in Preset

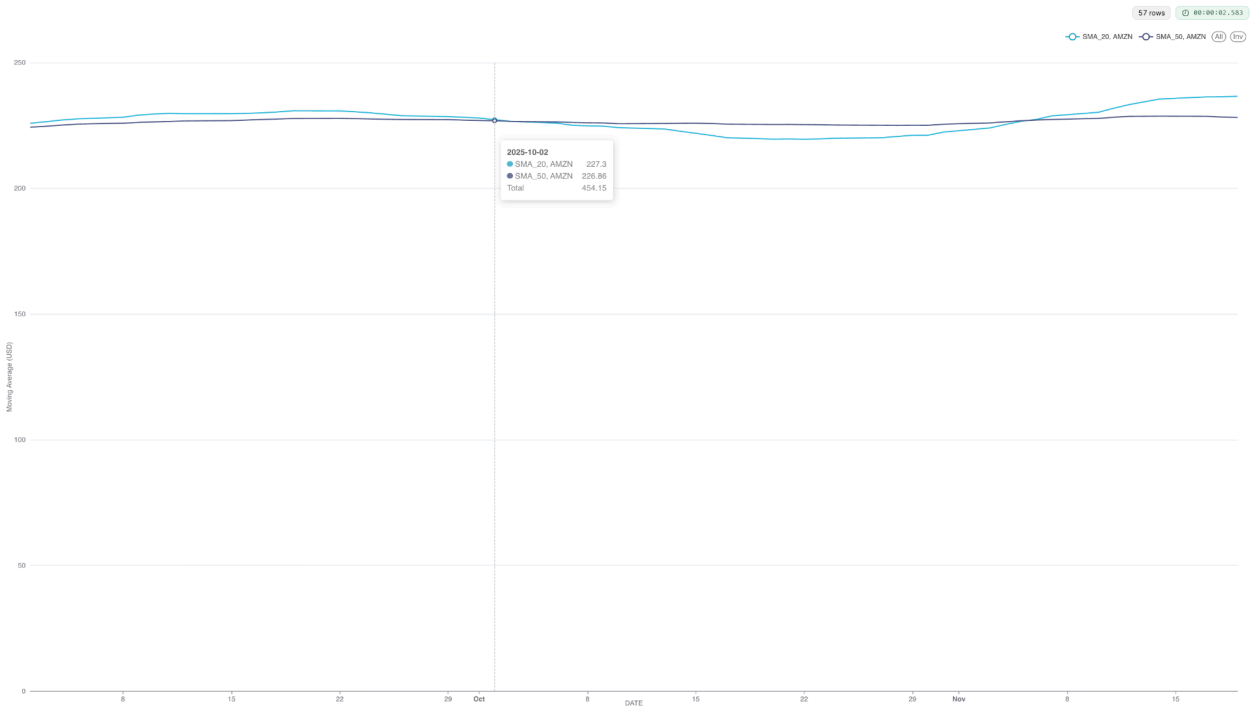
The final interactive dashboard was built in **Preset** using the dataset ANALYTICS.FINAL_STOCK_PRICE_TABLE from Snowflake. It integrates all four dbt-generated transformations—**Daily Returns**, **Rate of Change (ROC) and Momentum**, **Simple Moving Averages (SMA 20 & 50)**, and **20-Day Rolling Volatility**—to enable comparative stock analysis.

The dashboard contains four visualizations:

1. **Stock Price with SMA (20 & 50):**
   - This chart was created using the dataset ANALYTICS.FINAL_STOCK_PRICE_TABLE from Snowflake.
   - It is built as a **Line Chart** in Preset. The **X-axis** represents DATE (time grain: Day), and the **Y-axis** displays the calculated averages for SMA_20 and SMA_50.
   - The **Dimensions** field was set to SYMBOL to display separate lines for each stock ticker.
   - Filters were applied for **SYMBOL** (AAPL, MSFT, AMZN) and the **Date Range** (last 90 days).
   - The **X-axis label** is "Date," and the **Y-axis label** is "Stock Price / Moving Average (USD)."
   - This chart shows short- and medium-term price trends for each company, making it easy to observe crossovers between short-term and long-term averages.



For a single ticker

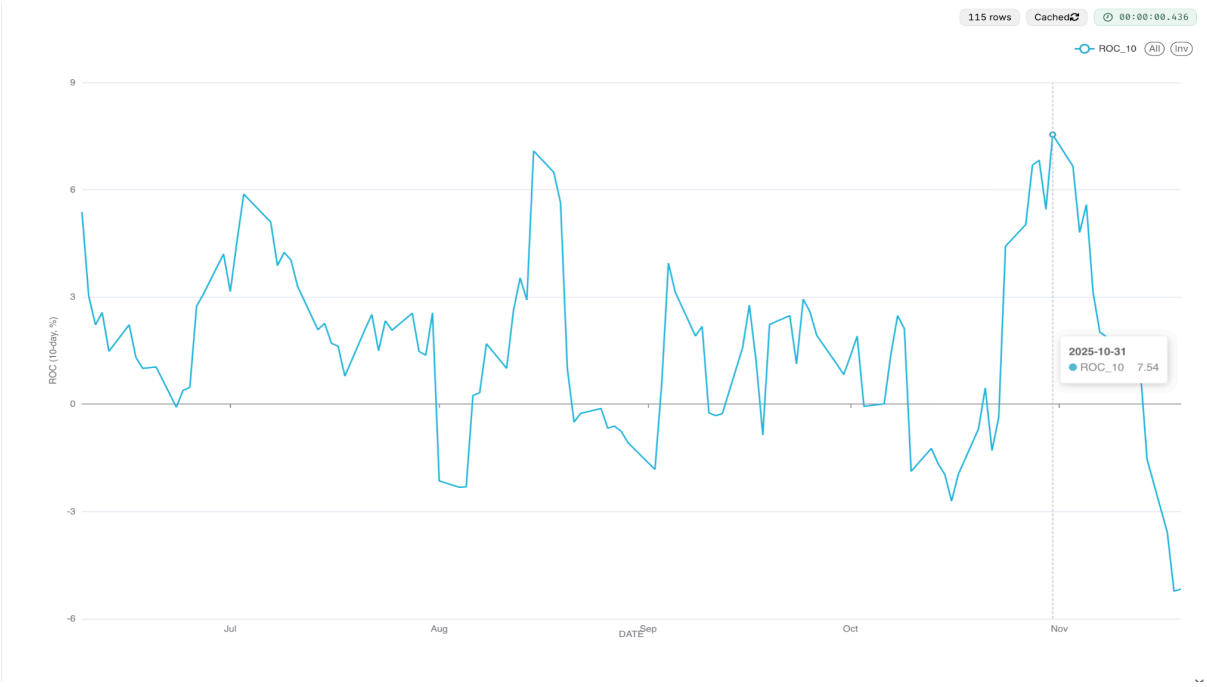


2. Daily Returns (%)

- The Daily Returns chart uses the same dataset ANALYTICS.FINAL_STOCK_PRICE_TABLE and is built as a **Time-Series Line Chart** in Preset.
- The **X-axis** represents DATE (time grain: Day), and the **Y-axis** displays AVG(DAILY_RETURN_PCT) to show the daily percentage change in price.
- The **Dimensions** field was set to SYMBOL, allowing each stock's return to be shown as a separate colored line.
- Filters were applied for **SYMBOL** (AAPL, MSFT, AMZN) and a **Date Range** of the last 90 days.
- The **X-axis** is labeled "Date," and the **Y-axis** is labeled "Daily Return (%)."
- This visualization highlights daily price fluctuations, making it easy to compare short-term performance and identify volatility patterns across the three stocks.

3. Rate of Change (ROC) and Momentum

- This visualization combines two technical indicators—**Rate of Change (ROC)** and **Momentum**—using the dataset ANALYTICS.FINAL_STOCK_PRICE_TABLE.
- It was built as a **Mixed Line and Bar Chart** in Preset, with **DATE** on the X-axis and metrics AVG(ROC_10) (line) and AVG(MOMENTUM_10) (bar) on the Y-axis.
- The **Dimensions** field was set to SYMBOL so that each stock's ROC and Momentum values appear distinctly.
- Filters were applied for **SYMBOL** (AAPL, MSFT, AMZN), excluding NULL values, and a **Date Range** of the last 90 days.
- The **X-axis** is labeled "Date," and the **Y-axis** is labeled "ROC (%) / Momentum (Price Units)."
- This chart displays both the speed (ROC) and the strength (Momentum) of price movements, providing valuable insight into each company's market direction and performance momentum over time.



4. Volatility (20-Day Rolling Standard Deviation)

- The final chart visualizes the 20-Day Rolling Volatility, derived from the same dataset ANALYTICS.FINAL_STOCK_PRICE_TABLE.
- It was configured as a Line Chart in Preset, with DATE on the X-axis and AVG(VOL_20D) on the Y-axis.
- The Dimensions field was set to SYMBOL, allowing each stock's volatility to be displayed as a separate line.
- Filters were applied for SYMBOL (AAPL, MSFT, AMZN) and a Date Range of the last 90 days.
- The X-axis is labeled "Date," and the Y-axis is labeled "Volatility (20-Day Rolling Std Dev)."
- This chart illustrates the 20-day rolling standard deviation of daily returns, providing insights into price

stability and risk variation over time for each company.

VII Testing and Idempotency

Testing and idempotency checks were performed to ensure the system's reliability and data integrity. Idempotency was verified in the ETL DAG by re-running it for the same date and confirming no duplicates in RAW.STOCK_PRICES via a Snowflake query (e.g., SELECT COUNT() FROM RAW.STOCK_PRICES GROUP BY symbol, date HAVING COUNT() > 1 returned empty).

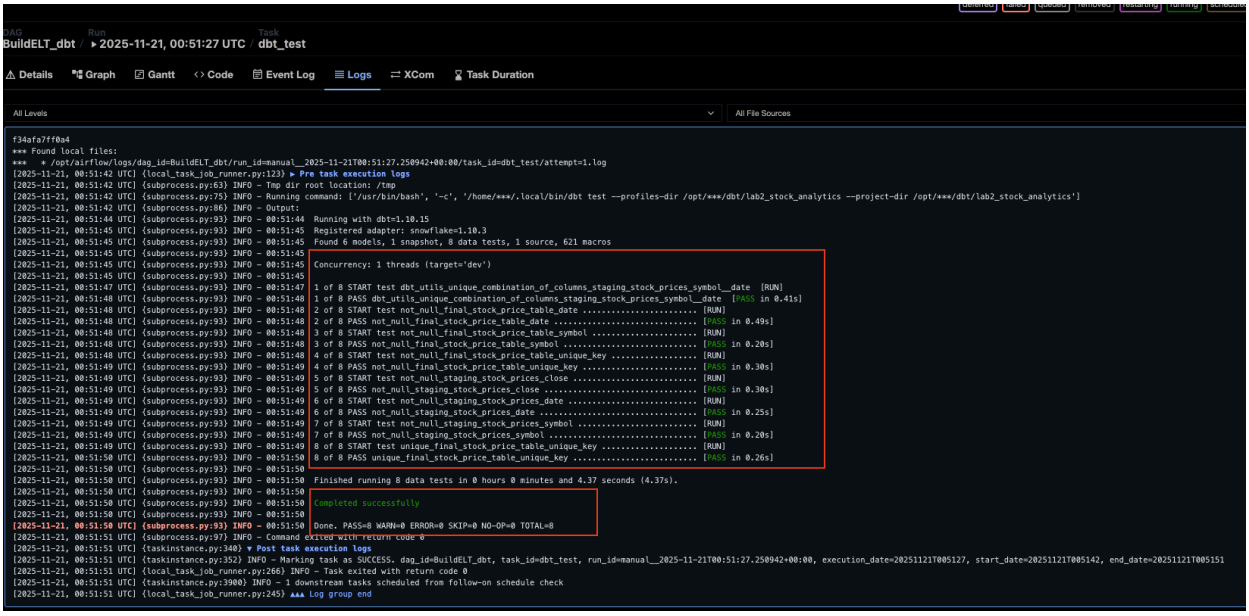
Idempotency code

```
        # Delete overlapping rows (idempotent load)
        cur.execute(f"""
            DELETE FROM {database}.{schema}.{table}
            WHERE symbol = '{symbol}' AND date BETWEEN '{min_d}' AND '{max_d}'
        """)
        # Load new data
        populate_table_via_stage(cur, database, schema, table, file_path)
        cur.execute("COMMIT;")
        print(f"Loaded {symbol}: {min_d}..{max_d}")
    except Exception as e:
        cur.execute("ROLLBACK;")
        print(f"Load failed for {symbol}: {e}")
        raise
    finally:
        # Remove temp file
        try:
            os.remove(file_path)
        except FileNotFoundError:
            pass
```

After rerunning the pipeline and checking the count should return empty if idempotent

Dbt run log :

All tests are passed



VIII Airflow Variables and Connections

Variables

- stock_symbols – List of tickers for dynamic ETL task generation.
- snowflake_db, snowflake_schema, snowflake_table – Target Snowflake locations for ETL loads.
- etl_table, train_view, forecast_table, final_table – Table/view paths for ML forecasting.
- forecast_fn – Name of the Snowflake ML forecast model.
- snowflake_conn_id – Stores "my_snowflake_conn" (the name of the Snowflake connection).

Connection

- my_snowflake_conn – Airflow Snowflake connection storing account, credentials, warehouse, role, and schema; accessed via SnowflakeHook.

IX. Conclusion

This project successfully demonstrated the design and implementation of a complete, automated stock-price analytics and forecasting pipeline using **Snowflake**, **Apache Airflow**, **dbt**, and **Preset**.The **ETL layer** ingested reliable market data from *yfinance* into Snowflake through Airflow, ensuring transactional integrity and idempotent loading.The **ML Forecast pipeline** applied Snowflake's native forecasting capabilities to generate seven-day predictions for multiple tickers.
The **ELT workflow** developed in dbt transformed the raw data into structured layers—*Staging*, *Intermediate*, and *Analytics*—producing key indicators such as *Daily Returns*, *ROC & Momentum*, *Moving Averages*, and *Volatility*.
Finally, the **Preset dashboard** provided an interactive interface to visualize these indicators, enabling dynamic trend analysis and comparison across stocks.

Together, these components illustrate an end-to-end cloud-based data-engineering solution that is scalable, maintainable, and analytics-ready. The system can easily be extended to include additional tickers, advanced ML models, or real-time streaming data sources in future work.

The complete project code, including all Airflow DAGs, dbt models, snapshots, and configuration files, is available in the public GitHub repository:

https://github.com/rishitha0547/DATA226/tree/main/DATA226_LAB2

X. References

1. Apache Airflow Documentation. *Apache Software Foundation*, 2025. Available: https://airflow.apache.org/

2. Snowflake Documentation. *Snowflake Inc.*, 2025. Available: https://docs.snowflake.com/

3. dbt Documentation. *dbt Labs*, 2025. Available: https://docs.getdbt.com/

4. Preset (Apache Superset) Documentation. *Preset Inc.*, 2025. Available: https://preset.io/docs/

5. Yahoo Finance API (yfinance) Python Library. *GitHub Repository*, 2025. Available: https://github.com/ranaroussi/yfinance

6. Snowflake ML Forecasting Guide. *Snowflake Inc.*, 2025. Available: https://docs.snowflake.com/en/user-guide/ml-forecast